TP 16 : Tas binomiaux

Ce TP est à réaliser dans son entièreté en C.

Préliminaires

Un arbre binomial est défini récursivement de la façon suivante :

- l'arbre binomial d'ordre 0 est un simple nœud.
- l'arbre binomial d'ordre k possède une racine de degré k et ses fils sont des arbres binomiaux d'ordre $k-1$, $k-2, \ldots, 2, 1, 0$. Ceux-ci sont ordonnés par ordre croissant.

Un arbre binomial d'ordre k peut aussi être construit à partir de deux arbres d'ordre $k-1$ en faisant de l'un des deux le fils le plus à gauche de la racine de l'autre. Un arbre binomial d'ordre k possède 2^k nœuds, et a pour hauteur k. Vous pouvez essayer de le démontrer à partir de la définition si vous le souhaitez.

Un tas binomial est un ensemble d'arbres binomiaux satisfaisant certaines propriétés. Ce n'est donc absolument pas un arbre binaire presque complet comme ce qu'on a pu voir en cours et l'on fera attention à ne pas mélanger les deux notions.

On propose d'utiliser un type récursif simple pour implémenter les arbres composant un tas binomial. Chaque nœud est étiqueté à l'aide de 3 entiers : sa clef, sa valeur et l'ordre de l'arbre binomial enraciné en celui-ci.

```
typedef struct arbre =
{
    int clef;
    int valeur;
    int ordre;
    struct arbre* liste_enfants;
    struct arbre* suivant;
}arbre_binomial;

typedef arbre_binomial* tas_binomial;
```

Chaque objet de type `arbre_binomial` est en réalité un maillon d'une liste chaînée d'arbres binomiaux. L'attribut `suivant` est un pointeur vers le maillon suivant, celui de l'arbre suivant la fratrie tandis que l'attribut `liste_fils` est une autre liste chaînée d'arbres binomiaux, les enfants du nœud considéré. La structure de tas binomial correspond simplement à une liste chaînée d'arbres, l'attribut `suivant` permettant de faire le lien entre les différents éléments de la liste.

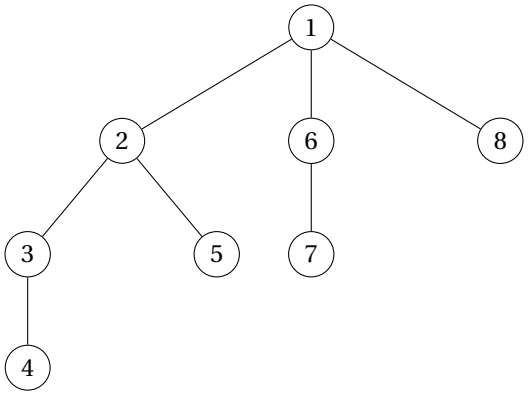

FIGURE 1 – Arbre binomial d'ordre 3

Sur l'exemple de la figure 1, on a représenté un arbre binomial d'ordre 3. L'attribut `suivant` du nœud de clef 2 pointe vers celui de clef 6 tandis que sont attribut `liste_enfants` pointe vers le nœud de clef 3.

Question 1 : Écrire une fonction `arbre_binomial* creer_noeud(int clef, int valeur, int ordre)` qui alloue l'espace mémoire nécessaire pour un nœud d'arbre binomial et initialise les attributs fournis. Les autres seront initialisés à `NULL`.

Question 2 : Écrire une fonction `arbre_binomial* exemple(int n)` qui, étant donné un entier n, renvoie un arbre binomial d'ordre n dont toutes les clefs et valeurs valent 1.

Tas binomiaux

Les arbres composant un tas binomial vérifient les propriétés supplémentaires suivantes :

- La propriété de tas-minimum, c'est-à-dire que la clef d'un nœud est toujours inférieure ou égale à celle de ses fils. La clef minimum d'un tas binomial se trouve donc à la racine de l'un de ses arbres.
- Pour chaque entier j, il existe au plus un arbre binomial d'ordre j au sein du tas binomial.
- Les arbres binomiaux sont classés selon leur ordre croissant.

Néanmoins, les opérations que l'on souhaite réaliser sur cette structure de données sont similaires :

- **Insertion**
- **Extraire le minimum**
- **Décroître une clef**
- **Unir deux tas**

Question 3 : Écrire une fonction `void liberer_tas(tas_binomial t)` qui, étant donné un tas binomial, libère l'espace mémoire alloué pour celui-ci. On veillera à bien libérer l'espace mémoire utilisé par chacun des arbres.

Réaliser l'union

Comme son nom ne l'indique pas du tout, l'union est l'opération la plus complexe à implémenter sur les tas binomiaux. En effet, on verra que les autres y font appel. L'union ressemble en quelque sorte à la fusion de deux listes triées dans le tri fusion. On parcours les deux listes en parallèle dans l'ordre croissant et l'on construit un arbre d'ordre $j+1$ à chaque fois que l'on réalise que l'on a deux arbres d'ordre j.

Question 4 : Écrire une fonction `arbre_binomial* fusion(arbre_binomial a1, arbre_binomial a2)` qui, étant donnés deux arbres binomiaux de même ordre j respectant la propriété de tas-minimum, renvoie un arbre binomial d'ordre $j+1$ respectant la propriété de tas-minimum qui contient exactement les mêmes éléments que l'union des éléments des deux arbres initiaux. On pourra utiliser l'une des définitions possibles des arbres binomiaux.

Question 5 : Écrire une fonction `tas_binomial fusionner_doublons(tas_binomial t)` qui, étant donné une instance de `tas_binomial` où la seconde condition n'est pas vérifiée, réalise un parcours de la liste en fusionnant les arbres dont les ordres sont identiques. On veillera à conserver les autres conditions.

Question 6 : Écrire une fonction `tas_binomial unir(tas_binomial t1, tas_binomial t2)` qui réalise l'union des tas binomiaux passés en paramètre.

Autres opérations

Question 7 : Écrire une fonction `tas_binomial insertion(tas_binomial t1, int clef, int valeur)` qui, étant donné un tas binomial, une clef et la valeur associée, insère celles-ci dans le tas. On pourra chercher à utiliser la fonction précédente sur des tas binomiaux bien choisis.

Question 8 : Écrire une fonction `int trouver_min(tas_binomial t1)` qui étant donné un tas binomial, renvoie la valeur dont la clef est minimale. On utilisera la propriété de tas-minimum que possèdent tous les arbres binomiaux composant le tas.

Question 9 : Écrire une fonction `tas_binomial extraire_min(tas_binomial t)` qui étant donné un tas binomial, renvoie le tas binomial dont le couple de clef minimale a été retiré.

Question 10 : Expliquer pourquoi il est compliqué d'implémenter l'opération de diminution de la clef avec un tel choix d'implémentation de structure de données.

Question 11 : Quitte à modifier certaines choses, proposer un nouveau type permettant d'implémenter une telle méthode.

Question 12 : Expliquer comment il est alors possible de retirer un élément arbitraire dans un tas binomial.

Pour aller plus loin

Les tas de Fibonacci sont une amélioration des tas binomiaux qui permettent d'avoir une union de deux tas plus efficace. Si cela vous intéresse, vous pouvez regarder cette vidéo : `https://www.youtube.com/watch?v=6JxvKfSV9Ns`. C'est en anglais et ça fait réviser (un peu) la complexité amortie.